

Q34. Minimum Set Cover Problem (NP-Complete Problem)

1. Aim

To solve the Minimum Set Cover Problem using a **backtracking approach** with pruning and determine the minimum number of subsets required to cover all elements of a given universe.

2. Problem Statement

Given a universe of elements and a collection of subsets, find the smallest number of subsets whose union contains every element in the universe.

The problem is **NP-Complete**, so the backtracking method explores possible combinations while pruning unnecessary choices to reduce the search space.

3. Objectives

- Find a minimum collection of subsets covering the complete universe.
- Apply backtracking to explore possible subset selections.
- Use pruning to eliminate redundant or unnecessary subsets.
- Verify that all elements are covered.
- Analyze the time and space complexity.

4. Input Details

Universe:

$U = \{1, 2, 3, 4, 5, 6\}$

Subsets:

$S1 = \{1, 2, 3\}$

$S2 = \{2, 4\}$

$S3 = \{3, 4, 5\}$

$S4 = \{4, 5, 6\}$

$S5 = \{1, 6\}$

5. Constraints

- Every element in the universe must be covered.
- A subset can either be selected or not selected.
- The same subset cannot be selected more than once.
- The objective is to minimize the number of selected subsets.
- A solution is feasible only when:

Union of selected subsets = Universe

6. Algorithm Used

Backtracking with Pruning

The algorithm considers each subset as either selected or not selected. It recursively explores the possible combinations and keeps track of the elements already covered.

Pruning is applied when:

- The current number of selected subsets is already greater than the best solution.
- A subset does not contribute any new uncovered elements.
- All elements are already covered.

7. Algorithm

1. Start with an empty set of covered elements.
2. Start with zero selected subsets.
3. Select a subset and add its elements to the covered set.
4. Recursively continue with the remaining subsets.
5. If all universe elements are covered, update the best solution.
6. If the current solution is already larger than the best solution, stop exploring that branch.
7. Ignore subsets that do not add any new element.
8. Continue until all possible useful combinations are checked.
9. Return the smallest valid collection of subsets.

PROGRAM:

```
1 def set_cover(universe, subsets):
2     best_solution = []
3
4     def backtrack(index, covered, selected):
5         nonlocal best_solution
6
7         # Check if all elements are covered
8         if covered == universe:
9             if not best_solution or len(selected) < len
10                (best_solution):
11                 best_solution = selected.copy()
12                 return
13
14         # Pruning
15         if best_solution and len(selected) >= len
16            (best_solution):
17             return
18
19         # No more subsets
20         if index == len(subsets):
21             return
22
23         # Select current subset
24         new_elements = subsets[index] - covered
```

OUTPUT:

```
Minimum Set Cover Problem
-----
Universe: {1, 2, 3, 4, 5, 6}

Subsets:
S1: {1, 2, 3}
S2: {2, 4}
S3: {3, 4, 5}
S4: {4, 5, 6}
S5: {1, 6}

Selected Subsets:
S1 {1, 2, 3}
S4 {4, 5, 6}

Complete Coverage: {1, 2, 3, 4, 5, 6}
Minimum Number of Subsets: 2
```

10. Analysis

The algorithm finds that **S1 and S4** are sufficient to cover the complete universe.

$$\begin{aligned} S1 \cup S4 \\ &= \{1,2,3\} \cup \{4,5,6\} \\ &= \{1,2,3,4,5,6\} \end{aligned}$$

Therefore, the minimum number of subsets required is **2**.

11. Pruning Strategy

Pruning reduces unnecessary recursive searches.

- If all elements are already covered, the search stops.
- If the current solution contains as many subsets as the best solution, that branch is stopped.
- A subset that contributes no new elements is skipped.
- These conditions reduce the number of combinations that need to be examined.

12. Completeness of the Solution

The solution ensures complete coverage by checking:

covered == universe

Only a selection whose union contains every element of the universe is accepted as a valid solution.

Since the backtracking process considers both **selecting and not selecting** each useful subset, it can find the optimal minimum cover.

13. Complexity Analysis

For n subsets, there can be up to:

$$2^n$$

possible subset combinations.

Therefore:

- **Worst-case Time Complexity:** $O(2^n \times m)$

- **Space Complexity:** $O(n + m)$

where:

- n = number of subsets
- m = number of elements in the universe

Because Set Cover is NP-Complete, the number of possible combinations increases exponentially as the number of subsets grows.

14. Scalability

The backtracking method works well for **small and medium-sized problems**. For a large number of subsets, the exponential search space can become expensive.

Pruning improves practical performance by avoiding branches that cannot produce a better solution. However, it does not change the worst-case exponential complexity.

15. Result

The backtracking algorithm successfully finds the minimum set cover.

For the given input:

Minimum Cover = {S1, S4}

Minimum Number of Subsets = 2

Complete Coverage = Yes

16. Conclusion

The Minimum Set Cover Problem was solved using backtracking with pruning. The constraints ensure that every element is covered while minimizing the number of selected subsets. Pruning eliminates redundant branches and improves practical performance. Since the problem is NP-Complete, its worst-case complexity remains exponential, making the approach more suitable for smaller problem sizes.